Mini-simulador de memoria compartida distribuida a nivel de página en Linux

Armando, Daniel, Guillermo, Daniel y Mauricio
Sistemas Operativos — Proyecto 10
Catedrático: David Garcia Cerna
Institución y sección: por completar

***Resumen*—Se presenta un simulador educativo de memoria compartida distribuida, construido en C11 para Linux. Dos procesos independientes se comunican mediante una conexión TCP local y representan un espacio lógico de 1 MiB dividido en 256 páginas de 4096 bytes. El sistema detecta accesos a copias inválidas, transfiere páginas completas, migra propiedad y mantiene copias de lectura opcionales. Un coordinador central serializa las operaciones y exige invalidaciones confirmadas antes de conceder escritura. La implementación conserva módulos separados de nodo, transporte, memoria, protocolo e interfaz. Se verificó con tres programas de pruebas C y 22 escenarios de integración; nueve escenarios DSM también se ejecutaron con detectores de errores de memoria y comportamiento indefinido. Se incluyen trazas reproducibles y una demostración de migración e invalidación. Los fallos de página son simulados, y el sistema no ofrece recuperación tras caídas ni acceso transparente mediante punteros.**

***Index Terms*—DSM, coherencia, consistencia secuencial, páginas, TCP, Linux.**

I. Introducción

En un sistema distribuido, cada proceso dispone de memoria local y el intercambio de datos requiere comunicación. Este proyecto permite observar cómo una abstracción de memoria compartida puede construirse sobre mensajes sin que los participantes compartan físicamente el mismo arreglo. El objetivo es didáctico: hacer visibles las decisiones que permiten usar una página remota y modificarla con exclusividad.

La guía del Proyecto 10 solicita dos procesos conectados por sockets, un arreglo paginado, propiedad inicial concentrada en un nodo y una demo de migración e invalidación. Se permite ejecutarlos en la misma computadora. La solución utiliza Pop!_OS y conserva una arquitectura modular desarrollada previamente para un saludo TCP.

Las contribuciones del trabajo son un simulador ejecutable, un contrato de mensajes acotado, una CLI explícita y pruebas repetibles con evidencia. No se propone un nuevo algoritmo de coherencia ni se evalúa rendimiento de sistemas DSM de producción.

II. Fundamento conceptual

Li y Hudak estudiaron memoria virtual compartida organizada por páginas y gestores de coherencia centrales y distribuidos [1]. Nuestro modelo toma como antecedente la necesidad de mantener información sobre propiedad y copias, pero emplea comandos de acceso explícito. No implementa el sistema IVY ni acceso transparente a memoria virtual.

La coherencia de página y el orden de accesos son cuestiones distintas. La noción de consistencia secuencial de Lamport exige explicar los resultados por un orden global compatible con el orden de cada participante [2]. En nuestro diseño, la serialización del coordinador facilita razonar sobre operaciones completadas sin fallos.

TCP se utiliza como flujo confiable de bytes; no conserva por sí solo las fronteras de los mensajes de aplicación [3]. Por ello el transporte define una cabecera, acumula recepciones parciales y conserva offsets de envío. Los tratamientos de EOF, EAGAIN y SIGPIPE siguen las APIs documentadas por Linux [4], [5], [6].

III. Modelo y arquitectura

III-A. Nodos y recursos

El mismo binario se ejecuta con identificador 0 o 1. Ambos nodos crean un servidor en 127.0.0.1; únicamente N0 inicia la conexión y N1 la acepta. Esto evita conexiones TCP duplicadas. El arranque tolera cualquiera de los dos órdenes mediante hasta 30 intentos de conexión, separados por un segundo cuando el peer aún no escucha.

Cada nodo reserva con calloc un arreglo contiguo de $256 \times 4096 = 1048576$ bytes. El número y tamaño de páginas son parámetros del modelo, no páginas físicas consultadas al kernel. La página p comienza en el desplazamiento $p \times 4096$. Se valida el índice antes de acceder. N0 comienza como dueño de todas las páginas, inicializadas a cero; N1 mantiene todos sus metadatos en INVALID.

III-B. Módulos

Main valida configuración; node administra conexión, eventos y ciclo de vida; network codifica y transporta bytes; memory aplica operaciones locales; protocol coordina permisos y transacciones; cli analiza órdenes y presenta resultados. Memory no conoce sockets y la CLI no modifica directamente los bytes de una página.

Un solo hilo atiende red y entrada estándar mediante poll. La recepción incremental no espera de forma bloqueante el resto de una trama. Una cola de envío de 65536 bytes conserva el progreso cuando el socket no acepta más datos. Se limita cada lote de despacho a 32 mensajes para mantener capacidad de respuesta de la terminal.

III-C. Metadatos y estados

Cada página mantiene estado, versión y una indicación informativa del propietario. Los estados son INVALID, READ_ONLY y OWNER. Los bytes residuales de INVALID nunca satisfacen una lectura. OWNER identifica la copia autoritativa; no concede permiso permanente de escritura si existen lectores.

El directorio autoritativo reside en N0 y contiene propietario, versión confirmada y máscara de holders válidos. La máscara utiliza un bit por nodo. Las versiones aumentan una vez por escritura y no cambian al copiar o migrar. Se rechaza una escritura que desbordaría uint32. Las indicaciones locales de propietario pueden estar desactualizadas y no se usan como autoridad.

IV. Protocolo implementado

IV-A. Negociación y mensajes

Se conserva el saludo textual HELLO/HELLO_ACK y luego se negocian identidad y geometría mediante HELLO_CONFIG. La CLI se habilita tras esta negociación. Cada trama binaria contiene una cabecera de 40 bytes: diez enteros de 32 bits en orden de red, con magic, versión, tipo, origen, solicitud, página, propietario, versión de página, flags y longitud. No se transmiten estructuras C ni su padding.

Los mensajes principales son READ_REQUEST, WRITE_REQUEST, FETCH_PAGE, PAGE_DATA, GRANT, INVALIDATE, ACK y ERROR. PAGE_DATA lleva exactamente 4096 bytes. HELLO_CONFIG lleva 12; los restantes no llevan payload. Se comprueba la cabecera antes del cuerpo, con tipos y longitudes acotados. Un origen y un identificador creciente correlacionan cada solicitud; no se reutilizan identificadores después de un rechazo BUSY.

IV-B. Lectura con migración y copia opcional

La orden read cumple el flujo de la guía: si el solicitante necesita datos, se preserva una instantánea de la página, se invalidan los otros holders y se instala OWNER en el solicitante. Si el dueño es N1, N0 recupera primero los bytes con FETCH_PAGE. La instantánea no se registra como una copia de lectura del coordinador.

La orden separada read-copy mantiene al dueño y concede READ_ONLY al solicitante. Esta extensión permite demostrar el escenario condicional de lectores de la guía. No se presenta una réplica como si fuera migración de propiedad. Una lectura sobre copia válida evita transferir datos, pero continúa solicitando autorización a N0.

IV-C. Escritura y exclusividad

Una escritura obtiene primero los datos necesarios y después invalida a todos los holders distintos del solicitante. N0 envía INVALIDATE a N1 y espera ACK_INVALIDATED cuando la copia es remota; si la copia está en N0, se invalida directamente antes de conceder. Cuando N1 ya es dueño, envía INVALIDATE como solicitud de exclusividad al coordinador, que invalida su copia local antes de responder GRANT.

El solicitante modifica únicamente después de la concesión. El texto reemplaza toda la página: se borra el contenido anterior, se copian de 1 a 4095 bytes y se conserva terminación NUL. ACK_DONE comunica finalización y versión al coordinador, que actualiza el directorio. El estado estable vuelve a contener un único dueño.

IV-D. Transacciones y orden

La máquina global pasa por IDLE, FETCHING si hace falta, INVALIDATING y WAIT_DONE. No admite otra operación hasta recibir la finalización. Una solicitud adicional recibe BUSY y debe reintentarse manualmente. Las solicitudes de N0 recorren la misma lógica mediante despacho local, evitando reglas distintas para cada nodo.

Los invariantes estables son: un único OWNER por página, coincidencia de datos y versión entre copias válidas, escritura exclusiva y ausencia de lecturas INVALID. Durante una transferencia puede haber cero dueños locales; la instantánea y el bloqueo global impiden perder datos o servir el estado intermedio. No se afirma recuperación de esos invariantes después de una caída.

V. Errores y ciclo de vida

Los argumentos desconocidos, repetidos o incompletos se rechazan. Los puertos deben estar entre 1 y 65535 y ser distintos. El parser de comandos comprueba rango, longitud y tokens sobrantes; una orden inválida no modifica memoria. Los datos impresos se acotan y escapan para evitar interpretar bytes de control como salida de terminal.

El saludo y configuración tienen plazo absoluto de 10 segundos; una operación, trama parcial o cola pendiente dispone de 5 segundos. Estos plazos no se prolongan por recibir fragmentos aislados. EOF en reposo cierra normalmente, mientras EOF parcial, ACK incoherente o timeout abortan la sesión. MSG_NOSIGNAL permite manejar EPIPE como error ordinario. Enter, exit y EOF de entrada esperan la operación pendiente; señales de cierre solicitan aborto y liberación de recursos.

VI. Evaluación reproducible

VI-A. Entorno y metodología

Las pruebas se realizaron el 17 de octubre de 2026 en Pop!_OS 24.04 LTS, Linux 7.1.5 y GCC 13.3, con C11 y advertencias Wall, Wextra y Wpedantic. Se compiló desde limpio, sin warnings. Las comprobaciones se separan en tres ejecutables C y pruebas Python que controlan procesos C reales sobre TCP. Python no sustituye el simulador.

Los tests C verifican límites y permisos de memoria, parser, framing y colas. La integración incluye ambos órdenes de arranque, migraciones, copias, datos actualizados, longitudes máximas y errores del peer. Un peer controlado retrasa ACK para comprobar que no se escribe antes de confirmar invalidaciones y provocar BUSY de forma determinista. Un oráculo secuencial contrasta 48 operaciones con semilla 10.

La suite completa tomó 48.166 segundos en la ejecución conservada. Esta duración incluye esperas intencionales de

Tabla I
RESULTADOS DE VERIFICACIÓN

Comprobación	Resultado
Compilación limpia	Sin warnings
Programas de prueba C	3 aprobados
Integración completa	22 aprobados
Escenarios DSM con ASan/UBSan	9 aprobados
Demo automática	Aprobada

Tabla II
CONTADORES AL TERMINAR LA DEMO

Métrica	N0	N1
Fallos simulados	2	1
PAGE_DATA enviados	1	2
PAGE_DATA recibidos	2	1
Invalidaciones aplicadas	2	1
Commits del coordinador	5	—

timeout y reintento y no representa latencia DSM. AddressSanitizer y UndefinedBehaviorSanitizer no detectaron errores en las ejecuciones indicadas. Esto es evidencia experimental y no una demostración formal exhaustiva.

VI-B. Demostración y datos observados

La demo realiza cinco accesos sobre página 3: escritura N0, lectura migratoria N1, copia de lectura N0, escritura N1 e invalidación de N0, y lectura migratoria de vuelta a N0. El texto final es el escrito por N1; la versión pasa de 0 a 1 y luego a 2. Se conservan logs de ambos nodos, comandos, entorno y SHA-256 del código ejecutado.

Se transfieren tres páginas completas, 12288 bytes de payload de datos, además de cabeceras y control. La segunda escritura no retransmite la página porque N1 ya es dueño. Los contadores de envío representan mensajes encolados; la demo comprueba su recepción, pero no deben interpretarse como confirmación ante fallos.

VII. LIMITACIONES Y CONCLUSIONES

El sistema cubre los flujos exigidos: dos procesos, detección de ausencia de página, transferencia completa, instalación, migración y exclusividad con invalidación. La separación modular y las trazas permiten relacionar cada comando con cambios observables de estado y mensajes de control.

Se limita a dos nodos en loopback, memoria volátil y una operación activa. No hay acceso transparente, persistencia, autenticación, recuperación ni reconexión. Tampoco se mide rendimiento sobre LAN ni se exploran todos los interleavings posibles. El coordinador es un punto único de fallo. Como trabajo futuro se proponen exploración de estados, experimentos de red y recuperación diseñada explícitamente, sin asumir que una reconexión simple preservaría la propiedad.

REFERENCIAS

[1] K. Li and P. Hudak, "Memory Coherence in Shared Virtual Memory Systems," *ACM Trans. Comput. Syst.*, vol. 7, no. 4, pp. 321–359, Nov. 1989, doi: 10.1145/75104.75105.

[2] L. Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," *IEEE Trans. Comput.*, vol. C-28, no. 9, pp. 690–691, Sept. 1979.

[3] Linux man-pages, "tcp(7)." [Online]. Available: https://man7.org/linux/man-pages/man7/tcp.7.html. Accessed: Oct. 17, 2026.

[4] Linux man-pages, "recv(2)." [Online]. Available: https://man7.org/linux/man-pages/man2/recv.2.html. Accessed: Oct. 17, 2026.

[5] Linux man-pages, "send(2)." [Online]. Available: https://man7.org/linux/man-pages/man2/send.2.html. Accessed: Oct. 17, 2026.

[6] Linux man-pages, "poll(2)." [Online]. Available: https://man7.org/linux/man-pages/man2/poll.2.html. Accessed: Oct. 17, 2026.